数据结构与算法知识点整理

目录

1 算法分析的数学基础 **2**

1.1 算法效率与增长的阶 . . . 2

1.2 渐进记号 . . . 2

1.2.1 同阶函数集合 $\Theta(g(n))$ . . . 2

1.2.2 渐近上界函数集合 $O(g(n))$ . . . 2

1.2.3 渐近下界函数集合 $\Omega(g(n))$ . . . 2

1.2.4 严格低阶与严格高阶 . . . 3

1.3 和式估计与界限 . . . 3

1.4 递归方程 . . . 3

1.4.1 替换方法 . . . 4

1.4.2 递归树方法 . . . 4

1.4.3 Master 定理 . . . 4

2 摊还分析 **5**

2.1 基本思想 . . . 5

2.2 聚集方法 . . . 5

2.2.1 栈操作 . . . 5

2.2.2 二进制计数器 . . . 5

2.3 会计方法 . . . 6

2.3.1 栈操作的会计分析 . . . 6

2.3.2 二进制计数器的会计分析 . . . 6

2.4 势能方法 . . . 6

2.4.1 栈操作的势能 . . . 7

2.4.2 二进制计数器的势能 . . . 7

2.5 动态表操作的摊还分析 . . . 7

2.5.1 表的扩张 . . . 7

2.5.2 表的收缩 . . . 8

2.5.3 动态表伪代码 . . . 8

2.5.4 动态表势能函数 . . . 9

3 期末相关例题 **10**

3.1 例题 1：循环复杂度 10

3.2 例题 2：对数复杂度 11

3.3 例题 3：Master 定理 11

3.4 例题 4：不能直接套用 Master 定理的情形 11

3.5 例题 5：栈的均摊复杂度 12

3.6 例题 6：动态数组扩容为 3 倍 13

1 算法分析的数学基础

1.1 算法效率与增长的阶

比较算法效率时，不直接依赖某一次程序运行时间。实际运行时间会受到程序语言、代码优化程度、机器运算速度和指令集等因素影响，因此用输入规模增大时运行时间的增长率来描述算法效率。

- 算法增长的阶也称为增长率或增长量级。
- 渐进效率关注输入规模足够大时的增长趋势。
- 分析时忽略低阶项和常数系数，只保留最高阶增长量。
- 常见增长阶包括：$O(1)$、$O(\log n)$、$O(n)$、$O(n\log n)$、$O(n^2)$、$O(2^n)$ 等。

1.2 渐进记号

1.2.1 同阶函数集合 $\Theta(g(n))$

$\Theta(g(n))$ 表示与 $g(n)$ 同阶的函数集合。若存在正常数 c_1, c_2 和 n_0，使得对所有 $n \geq n_0$，

$$0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n),$$

则

$$f(n) = \Theta(g(n)).$$

含义：$g(n)$ 同时给出 $f(n)$ 的渐近上界和渐近下界。

1.2.2 渐近上界函数集合 $O(g(n))$

$O(g(n))$ 表示不高于 $g(n)$ 阶的函数集合（渐近上界）。若存在正常数 c 和 n_0，使得对所有 $n \geq n_0$，

$$0 \leq f(n) \leq cg(n),$$

则

$$f(n) = O(g(n)).$$

含义：$g(n)$ 是 $f(n)$ 的渐近上界。

1.2.3 渐近下界函数集合 $\Omega(g(n))$

$\Omega(g(n))$ 表示不低于 $g(n)$ 阶的函数集合（渐近下界）。若存在正常数 c 和 n_0，使得对所有 $n \geq n_0$，

$$0 \leq cg(n) \leq f(n),$$

则

$$f(n) = \Omega(g(n)).$$

含义：$g(n)$ 是 $f(n)$ 的渐近下界。

1.2.4 严格低阶与严格高阶

- $f(n) = o(g(n))$：$f(n)$ 相对于 $g(n)$ 是严格低阶。
- $f(n) = \omega(g(n))$：$f(n)$ 相对于 $g(n)$ 是严格高阶。

常用判断方式：

$$\lim_{n\to\infty} \frac{f(n)}{g(n)} = \begin{cases} 0, & f(n) = o(g(n)), \\ c > 0, & f(n) = \Theta(g(n)), \\ \infty, & f(n) = \omega(g(n)). \end{cases}$$

注意：极限比值法只在极限存在（或趋于无穷）时可直接使用；若极限不存在（如振荡），不能据此断言渐近关系，需回到定义判断。

1.3 和式估计与界限

循环复杂度经常转化为和式估计。常用和式包括：

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2} = \Theta(n^2),$$

$$\sum_{i=0}^{k} 2^i = 2^{k+1} - 1 = \Theta(2^k),$$

$$\sum_{i=1}^{n} \frac{1}{i} = \Theta(\log n).$$

求和常用方法：

- 线性和：直接使用等差数列公式。
- 级数：区分算术级数与几何级数。
- 数学归纳法：先猜测界，再证明界成立。
- 分裂求和：把复杂区间拆成若干容易估计的区间。
- 积分近似：用积分估计单调函数和式的上下界。

1.4 递归方程

递归算法的复杂度通常写成递归方程。求解递归方程要先明确初始条件或边界条件。

1.4.1 替换方法

替换方法的基本步骤：

1. 猜测递归方程的渐进界。
2. 用数学归纳法证明猜测。
3. 若归纳步骤无法闭合，可在猜测中减去低阶项，使归纳条件更强。

关键点：猜测正确不代表能直接证明出来，有时需要处理向下取整、向上取整或低阶项。

1.4.2 递归树方法

递归树方法把递归方程展开成树：

- 内部结点表示除求解子问题外的代价，即分解或合并子问题的代价。
- 树的分支数由子问题数量决定。
- 叶结点表示边界条件的代价。
- 总代价等于递归树所有层代价之和。

递归树常用于生成合理猜测，再用替换方法证明猜测是否正确。

1.4.3 Master 定理

对形如

$$T(n) = aT(n/b) + f(n)$$

的递归方程，其中 $a \geq 1$、$b > 1$，比较 $f(n)$ 与 $n^{\log_b a}$：

1. 若 $f(n) = O(n^{\log_b a - \epsilon})$，则
$$T(n) = \Theta(n^{\log_b a}).$$
2. 若 $f(n) = \Theta(n^{\log_b a} \log^k n)$ 且 $k \geq 0$，则
$$T(n) = \Theta(n^{\log_b a} \log^{k+1} n).$$
注意 k 不能为负数；当 $f(n) = \Theta(n^{\log_b a}/\log n)$（即 $k = -1$）时结果为 $\Theta(n^{\log_b a} \log\log n)$，不能直接套用上式。
3. 若 $f(n) = \Omega(n^{\log_b a + \epsilon})$（存在常数 $\epsilon > 0$），且满足正则条件（存在常数 $c < 1$，使对所有充分大的 n 有 $af(n/b) \leq cf(n)$），则
$$T(n) = \Theta(f(n)).$$

Master 定理只覆盖三种标准情况；不属于三种情况的递归式需要用其他方法求解。

2 摊还分析

2.1 基本思想

摊还分析研究一系列数据结构操作的总时间，并把总时间平均摊到每次操作上。它关注的是任意操作序列的总成本上界。

- 有的操作代价很高，有的操作代价一般，有的操作代价很低。
- 摊还分析把总成本摊到每个操作上，得到单次操作的均摊代价。
- 摊还分析不涉及概率，不同于平均情况分析。

2.2 聚集方法

聚集方法直接估计一个长度为 n 的操作序列在最坏情况下的总代价 $T(n)$，再把

$$\frac{T(n)}{n}$$

作为每次操作的摊还代价。该方法不区分操作类型，序列中的每个操作被赋予相同的摊还代价。

2.2.1 栈操作

普通栈操作：

- `PUSH`：将对象压入栈。
- `POP`：弹出并返回栈顶元素，空栈时返回错误。

若再定义 `MULTIPOP`，一次弹出 k 个元素，实际弹出数为 $\min(k, s)$，其中 s 是当前栈中元素个数。虽然单次 `MULTIPOP` 最坏可能很贵，但在任意操作序列中，每个元素最多被压入一次、弹出一次，所以总弹出次数不超过总压入次数。于是 n 次操作总代价为 $O(n)$，单次摊还代价为 $O(1)$。

2.2.2 二进制计数器

二进制计数器执行 `INCREMENT` 时，最低位开始连续把若干个 1 翻成 0，再把遇到的第一个 0 翻成 1。单次操作最坏可能翻转多位，但在一系列递增操作中，低位频繁翻转，高位翻转次数少。总代价可以按每一位翻转次数求和，从而得到每次 `INCREMENT` 的摊还代价为 $O(1)$。

2.3 会计方法

会计方法为不同类型的操作分配不同的摊还代价。

- 如果某次操作的摊还代价大于实际代价，多余部分作为存款附加在数据结构的具体对象上。
- 如果某次操作的摊还代价小于实际代价，则用该次操作支付的摊还代价和对象上的存款共同支付实际代价。
- 只要任意操作序列上存款总额始终非负，则所有操作摊还代价之和就是实际代价之和的上界。

2.3.1 栈操作的会计分析

可令：

$$\widehat{c}(\texttt{PUSH}) = 2, \quad \widehat{c}(\texttt{POP}) = 0, \quad \widehat{c}(\texttt{MULTIPOP}) = 0.$$

PUSH 的 2 个单位中，1 个单位支付实际压栈代价，1 个单位作为存款附在被压入元素上。之后该元素被 POP 或 MULTIPOP 弹出时，用它自身的存款支付弹出代价。因此任意合理操作序列中存款非负，栈操作的摊还代价为 $O(1)$。

2.3.2 二进制计数器的会计分析

对位翻转定义：

$$0 \to 1 \text{ 的摊还代价为2}, \qquad 1 \to 0 \text{ 的摊还代价为0}.$$

$0 \to 1$ 时，1 个单位支付实际翻转代价，1 个单位作为存款附在该位上；之后该位从 $1 \to 0$ 时，用存款支付实际翻转代价。任意时刻存款余额等于计数器中 1 的个数，必然非负。因此每次 INCREMENT 的摊还代价为 $O(1)$。

2.4 势能方法

势能方法把会计方法中附着在具体元素上的存款，统一看作整个数据结构的势能。设第 i 次操作前后的数据结构为 D_{i-1}、D_i，实际代价为 c_i，势能函数为 Φ，则摊还代价定义为

$$\widehat{c_i} = c_i + \Phi(D_i) - \Phi(D_{i-1}).$$

对前 n 次操作求和：

$$\sum_{i=1}^{n} \widehat{c_i} = \sum_{i=1}^{n} c_i + \Phi(D_n) - \Phi(D_0).$$

若对所有 i 都有 $\Phi(D_i) \geq \Phi(D_0)$，则摊还代价总和是实际代价总和的上界。

2.4.1 栈操作的势能

可取势能函数为栈中元素个数：

$$\Phi(D_i) = |S_i|.$$

PUSH 实际代价为 1，势能增加 1，摊还代价为 2；POP 实际代价为 1，势能减少 1，摊还代价为 0；MULTIPOP 弹出 k' 个元素，实际代价为 k'，势能减少 k'，摊还代价为 0。故每次操作摊还代价为 $O(1)$。

2.4.2 二进制计数器的势能

可取势能函数为计数器中 1 的个数。一次 INCREMENT 若翻转 t 个 1 为 0，并翻转 1 个 0 为 1，则实际代价为 $t+1$，势能变化为 $1-t$，摊还代价为

$$(t+1) + (1-t) = 2.$$

因此单次 INCREMENT 的摊还代价为 $O(1)$。

2.5 动态表操作的摊还分析

2.5.1 表的扩张

动态表插入元素时：

- 若表中仍有空间，直接插入。
- 若表已满，则分配一个包含更多槽的新表，把原表各项复制到新表中，再插入新元素。

若每次扩张都分配一个比原表大一倍的新表，并且只执行插入操作，则装载因子始终至少为 1/2，浪费空间不超过总空间的一半。虽然触发扩张的插入操作实际代价较高，但一系列插入的均摊代价为 $O(1)$。

会计法中，可令每次 TABLE-INSERT 的摊还代价为 3：

- 1 个单位支付本次基本插入。
- 1 个单位作为自身存款。
- 1 个单位存入表中第一个尚无存款的数据项。

当发生扩张时，数据复制代价由数据项上的存款支付。任意时刻存款总额非负，所以均摊代价为 $O(1)$。

2.5.2 表的收缩

若装载因子低于 1/2 就立即收缩为原表的一半，可能导致插入和删除在临界位置反复触发扩张、收缩，造成高代价震荡。

改进策略：

- 删除元素时允许装载因子低于 1/2。
- 当删除使装载因子低于 1/4 时，将表缩小为原来的一半。
- 向满表插入时，仍将表扩大一倍。

这样装载因子保持在 1/4 到 1 之间：插入触发扩张后装载因子为 $(m+1)/(2m) > 1/2$（略高于 1/2，不是恰好 1/2）；仅在装载因子严格低于 1/4 时才收缩，收缩后装载因子约为 1/2（通常略低于 1/2）。

2.5.3 动态表伪代码

设动态表 T 有两个基本量：

$$T.num = \text{当前元素个数}, \qquad T.size = \text{当前表容量}.$$

装载因子定义为

$$\alpha(T) = \frac{T.num}{T.size}.$$

插入操作：

```
TABLE-INSERT(T, x)
    if T.size == 0
        allocate a table with 1 slot
        T.size = 1
    if T.num == T.size
        allocate a new table with 2 * T.size slots
        copy all T.num elements into the new table
        free the old table
        T.size = 2 * T.size
    T.table[T.num] = x
    T.num = T.num + 1
```

删除操作：

```
TABLE-DELETE(T, i)
    delete the element in position i
    move later elements left if the representation requires it
    T.num = T.num - 1
    if T.num > 0 and T.num < T.size / 4
        allocate a new table with T.size / 2 slots
```

```
        copy all T.num elements into the new table
        free the old table
        T.size = T.size / 2
```

注意: 下面的势能分析建立在“删除代价为常数”(例如只删表尾元素，或顺序存储允许空缺)的模型上。若在顺序存储中删除任意位置元素并整体左移，删除本身单次就是 $\Theta(n)$，不属于动态表扩缩摊还分析的范畴，不能把左移代价摊还成 $O(1)$。

2.5.4 动态表势能函数

对同时包含扩张和收缩的动态表，可取势能函数

$$\Phi(T) = \begin{cases} 2T.num - T.size, & \alpha(T) \geq \frac{1}{2}, \\ \frac{1}{2}T.size - T.num, & \alpha(T) < \frac{1}{2}. \end{cases}$$

该函数的具体取值含义如下:

- 当 $\alpha(T) = 1$ 时，$\Phi(T) = T.size$，势能最大，用于支付下一次扩张复制。
- 当 $\alpha(T) = 1/2$ 时，两段公式都给出 $\Phi(T) = 0$，扩张或收缩后势能回到低位。
- 当 $\alpha(T) = 1/4$ 时，低装载段给出 $\Phi(T) = T.size/4$，用于支付下一次收缩复制。
- 对任意状态都有 $\Phi(T) \geq 0$，因此总摊还代价是总实际代价的上界。

插入且不扩张时，实际代价 $c_i = 1$。若插入前后装载因子均不小于 $1/2$，则

$$\Delta\Phi = (2(T.num + 1) - T.size) - (2T.num - T.size) = 2,$$

所以

$$\widehat{c_i} = c_i + \Delta\Phi = 1 + 2 = 3.$$

若插入跨过 $1/2$ 的分界点，势能变化也被常数控制，因此摊还代价仍为 $O(1)$。

插入且扩张时，设扩张前 $T.num = T.size = m$。实际代价为复制 m 个旧元素再插入 1 个新元素:

$$c_i = m + 1.$$

扩张前

$$\Phi_{\text{before}} = 2m - m = m.$$

扩张后 $T.num = m + 1$、$T.size = 2m$，所以

$$\Phi_{\text{after}} = 2(m + 1) - 2m = 2.$$

于是

$$\widehat{c_i} = (m + 1) + (2 - m) = 3.$$

删除且不收缩时，实际代价为常数；势能变化也为常数，因此摊还代价为 $O(1)$。删除且收缩时，统一以下口径：设删除前 $T.num = n+1$、容量 $T.size = m$，本次删除 1 个元素后 $T.num = n$，且 $n < m/4$（触发收缩）。收缩需要把 n 个元素复制到容量 $m/2$ 的新表，实际代价为

$$c_i = 1 + n.$$

收缩前处于低装载段，删除前势能为

$$\Phi_{\text{before}} = \frac{m}{2} - (n+1).$$

收缩后 $T.num = n$、$T.size = m/2$，仍在低装载段，势能为

$$\Phi_{\text{after}} = \frac{m/2}{2} - n = \frac{m}{4} - n \geq 0.$$

代入摊还式

$$\widehat{c_i} = c_i + \Phi_{\text{after}} - \Phi_{\text{before}} = (1+n) + \left(\frac{m}{4} - n\right) - \left(\frac{m}{2} - (n+1)\right) = n + 2 - \frac{m}{4}.$$

由 $n < m/4$ 得 $\widehat{c_i} < 2$，单次删除的摊还代价仍为 $O(1)$。

3 期末相关例题

3.1 例题 1：循环复杂度

来源： 2025 哈尔滨工业大学（深圳）数据结构与算法试题，选择题第 7 题。

题目：

```
int sum = 0;
for (int i = 1; i < n; i *= 2)
    for (int j = 0; j < i; j++)
        sum++;
```

求时间复杂度。

解析： 外层循环中 i 的取值为

$$1, 2, 4, \ldots, 2^{k-1},$$

其中 $2^k \approx n$。内层循环分别执行 $1, 2, 4, \ldots, 2^{k-1}$ 次，因此总次数为

$$1 + 2 + 4 + \cdots + 2^{k-1} = 2^k - 1 = \Theta(n).$$

答案： $O(n)$，更精确地说为 $\Theta(n)$。

3.2 例题 2：对数复杂度

来源：课件随堂测试，算法分析部分。

题目：

```
void func(int n) {
    int i = 1;
    while (i <= n)
        i = i * 2;
}
```

求时间复杂度。

解析：设循环执行 t 次，则 $i = 2^t$。循环终止时 $2^t > n$，因此 $t = \lfloor \log_2 n \rfloor + 1$。

答案： $O(\log n)$。

3.3 例题 3：Master 定理

来源： 2025 哈尔滨工业大学（深圳）数据结构与算法试题，选择题第 6 题。

题目：已知

$$g(n) = 3g(n/4) + n, \qquad f(n) = n^2.$$

判断 $g(n)$ 与 $f(n)$ 的渐进关系。

解析：对 $g(n)$ 使用 Master 定理，$a = 3$，$b = 4$，非递归项为 n。有

$$n^{\log_b a} = n^{\log_4 3} \approx n^{0.792}.$$

因为 $n = \Omega(n^{\log_4 3 + \epsilon})$，且满足正则条件，所以

$$g(n) = \Theta(n).$$

与 $f(n) = n^2$ 比较，$g(n)$ 是 $f(n)$ 的严格低阶，因此

$$g(n) = O(f(n)).$$

答案： $g(n) = O(f(n))$。

3.4 例题 4：不能直接套用 Master 定理的情形

来源： 2025 秋哈尔滨工业大学（深圳）数据结构与算法期末考试试卷（回忆版），选择题第 10 题。

题目：下列递推式中，不能使用课内提到的 Master 定理三种情况直接解决的是：

$$T(n) = 2T(n/2) + n \log n.$$

解析：此时 $a = 2$，$b = 2$，临界函数为

$$n^{\log_b a} = n.$$

而

$$f(n) = n\log n = n^{\log_b a}\log^1 n.$$

即 $f(n) = \Theta(n^{\log_b a}\log^k n)$ 且 $k = 1 \geq 0$，恰好属于课内 Master 定理的第二种标准情况，可直接套用得到

$$T(n) = \Theta(n\log^2 n).$$

因此该递推式并非“不能直接套用”的答案；只有与临界函数相差的因子介于对数和多项式之间（如 $f(n) = n^{\log_b a}\log\log n$，不属于第二种情况）或正则条件不满足的情形，才需要其他方法。作为交叉验证，也可用递归树求最终结果：第 i 层有 2^i 个子问题，每个子问题规模为 $n/2^i$，该层非递归代价为

$$2^i \cdot \frac{n}{2^i}\log\frac{n}{2^i} = n(\log n - i).$$

递归深度为 $\log n$，所以总非递归代价为

$$\sum_{i=0}^{\log n-1} n(\log n - i) = n\sum_{j=1}^{\log n} j = \frac{n\log n(\log n+1)}{2} = \Theta(n\log^2 n),$$

与第二种情况结论一致。叶子层有 n 个规模为 1 的子问题，若 $T(1) = \Theta(1)$，叶子总代价为

$$\Theta(n).$$

因此

$$T(n) = \Theta(n\log^2 n) + \Theta(n) = \Theta(n\log^2 n).$$

答案：该递推式不能直接使用课内三种标准情况；最终正确结果为

$$T(n) = \Theta(n\log^2 n).$$

3.5 例题 5：栈的均摊复杂度

来源： 2025 哈尔滨工业大学（深圳）数据结构与算法试题，填空题第 8 题。

题目：在一个栈上定义两种操作：入栈代价为 1，连续出栈 k 个元素的代价为 $\min(k, s)$，其中 s 是执行出栈操作时栈中的元素个数。求最坏情况下单次操作的摊还复杂度。

解析：设一个长度为 n 的操作序列中，入栈次数为 p，连续出栈操作的总实际弹出元素数为 q。因为每个元素必须先入栈才能被弹出，且最多被弹出一次，所以

$$q \leq p.$$

入栈总代价为 p，连续出栈总代价为 q，于是总实际代价

$$C = p + q \leq 2p \leq 2n = O(n).$$

因此聚集方法给出的单次摊还代价为

$$\widehat{c} = \frac{C}{n} \leq 2 = O(1).$$

也可用势能方法。令势能函数为栈中元素个数：

$$\Phi(S) = |S|.$$

若执行 PUSH，实际代价 $c = 1$，势能增加 1：

$$\widehat{c} = 1 + (|S| + 1 - |S|) = 2.$$

若连续弹出 $r = \min(k, s)$ 个元素，实际代价 $c = r$，势能减少 r：

$$\widehat{c} = r + (|S| - r - |S|) = 0.$$

所以任意操作的摊还代价均为常数上界。

答案：单次操作摊还代价为 $O(1)$。

3.6 例题 6：动态数组扩容为 3 倍

来源： 26 春数据结构与算法期末，简答/大题第 4 题。

题目：顺序存储动态数组初始容量为 1，每次在表尾插入一个元素；当数组存满时扩容为原来的 3 倍，并复制原数组全部元素。普通插入代价为 1，复制一个元素代价也为 1。说明单次插入的均摊代价和均摊复杂度。

伪代码：

```
INSERT(A, x)
    if A.num == A.size
        new_size = 3 * A.size
        allocate B with new_size slots
        for i = 0 to A.num - 1
            B[i] = A[i]
        A = B
        A.size = new_size
    A[A.num] = x
    A.num = A.num + 1
```

注意：必须先记录新容量再替换数组指针，否则若写成“先 $A = B$ 再 $A.size = 3 \times A.size$”，会把容量元数据再乘一次 3，与 B 实际分配的空间不符。

解析一：聚集方法。设连续插入 n 次。扩容发生在容量为

$$1, 3, 9, 27, \ldots$$

时。设最后一次扩容前的容量为 3^k，则

$$3^k < n \le 3^{k+1}.$$

总复制代价至多为

$$1 + 3 + 9 + \cdots + 3^k = \frac{3^{k+1} - 1}{3 - 1} < \frac{3^{k+1}}{2} < \frac{3n}{2}.$$

普通插入总代价为 n，所以连续 n 次插入的总成本满足

$$C(n) \le n + \frac{3n}{2} = \frac{5n}{2} = O(n).$$

单次插入的均摊代价为

$$\widehat{c} = \frac{C(n)}{n} \le \frac{5}{2} = O(1).$$

解析二：势能函数。对“满时扩容为 3 倍”的只插入动态表，扩容后装载因子变为

$$\alpha = \frac{m+1}{3m} \approx \frac{1}{3}.$$

可取一个线性势能函数，让扩容后势能回到常数量级，满表时积累出 $\Theta(m)$ 势能支付复制。令

$$\Phi(T) = \frac{3}{2}T.num - \frac{1}{2}T.size.$$

当扩容后 $T.size = 3m$、$T.num = m+1$ 时，

$$\Phi_{\text{after}} = \frac{3}{2}(m+1) - \frac{1}{2}(3m) = \frac{3}{2}.$$

当下一次满表且 $T.num = T.size = 3m$ 时，

$$\Phi = \frac{3}{2}(3m) - \frac{1}{2}(3m) = 3m,$$

足以支付复制 $3m$ 个元素的代价。

若普通插入且不扩容，实际代价 $c = 1$，容量不变，元素数增加 1：

$$\Delta\Phi = \frac{3}{2}, \qquad \widehat{c} = 1 + \frac{3}{2} = \frac{5}{2}.$$

若满表扩容，设扩容前 $T.num = T.size = m$。实际代价为复制 m 个元素并插入新元素：

$$c = m + 1.$$

扩容前势能

$$\Phi_{\text{before}} = \frac{3}{2}m - \frac{1}{2}m = m.$$

扩容后 $T.num = m+1$、$T.size = 3m$：

$$\Phi_{\text{after}} = \frac{3}{2}(m+1) - \frac{1}{2}(3m) = \frac{3}{2}.$$

所以摊还代价为

$$\widehat{c} = (m+1) + \frac{3}{2} - m = \frac{5}{2}.$$

因此每次插入的摊还代价被常数 $\frac{5}{2}$ 控制。

答案：单次插入均摊代价为 $O(1)$。